We can search all the documents for shapes that fall in the geoshape enclosed in the envelope. The following query shows the code for this search.

Listing 12.13 Searching for all the shapes in a given envelope

```
GET myshapes/_search
{
  "query": {
    "shape":{
      "myshape": {
        "shape": {
          "type":"envelope",
          "coordinates":[[10,16],[14,10]]
        }
      }
    }
  }
}
```

Specifies a shape query

Field on which the query is run

Shape we want to construct

The envelope constructed by the given coordinates

The `shape` query defined in the listing searches for documents contained in the envelope created from the given coordinates `[10,16],[14,10]`. (Refer to figure 12.7 to see the bounded envelope created by this query and the shapes contained within it.)

As the query in listing 12.13 shows, we can create a polygonal envelope using the required coordinates (be sure the end coordinates meet, because open polygons are not supported in Elasticsearch). Keep in mind that `shape` queries are useful when working with 2D Cartesian coordinates for drawings and designs.

We now jump to a completely different set of queries under the umbrella of specialized queries: `span` queries. These queries support searching for terms at a particular location in a document, unlike normal search queries, where the token's position is ignored. This is best understood by running some queries; we do this next.

12.8 The span query

Term-level and full-text queries work the magic of searching at the token (word) level. They are not focused on the positions of the tokens (words) or their order. Consider the following text, a quote by Isaac Newton:

Plato is my friend. Aristotle is my friend. But my greatest friend is truth.

Suppose we want to find a document (quote) where Plato and Aristotle are both mentioned in the same order (not Aristotle and Plato), and the word *Aristotle* should be at least four positions away from *Plato*. Figure 12.8 shows this relationship: *Plato* is at position 1, *Aristotle* is at position 5, and their span is 5. Our requirement is to fetch all quotes where *Plato* and *Aristotle* meet this specification. We cannot satisfy this requirement using full-text (or term-level) queries. Although a prefix query can do a bit of justice to this requirement, it cannot satisfy other sophisticated criteria that we look at in the coming sections.